

The YouTube video you linked is from Symantec and shows how to set up a **high-risk isolation policy** in their Cloud Secure Web Gateway (SWG). It uses **Remote Browser Isolation (RBI)** — not the built-in site/process isolation inside Chrome or Edge.

How the isolation in the video works (and why it's "truly" isolated)

The risky website loads and runs entirely inside a disposable container/VM in Symantec's cloud (separate kernel and environment from your device).

You only receive a live video stream of the browser window (pixel rendering + input forwarding). Nothing executable, no files, no code, no plugins ever reach your local machine.

Even if malware runs, downloads files, or tries to exploit something inside that remote session, it stays trapped there — your host OS, kernel, and files are untouched.

Symantec can also block/sanitize downloads or force them through extra scanning. This is far stronger than local browser sandboxes.

Why Chrome/Edge site isolation isn't enough (and the news you saw is correct)

Chrome and Edge use **site isolation** (one renderer process per site + strong sandboxing). It's excellent for normal use and stops most attacks.

But everything still runs on your host kernel. Real-world sandbox escapes have happened (rare zero-days or chained exploits). Once the sandbox breaks, malware can reach your kernel, install itself, or get remote access.

That's exactly why enterprise RBI (like the video) or full VM isolation exists — they add a true kernel boundary.

Best isolation method for your project (separate kernel + VNC output)

You want **full kernel isolation** + separate render/frame buffer + VNC streaming. The strongest DIY option that matches your description is a **lightweight VM** (QEMU/KVM on Linux host, or VirtualBox). Containers (Docker) share the kernel, so skip those if you really need separation.

Top recommendation: **Tiny Core Linux (TinyCore variant)** inside a VM

This is the exact low-RAM GUI VM setup you couldn't find.

- ISO size: ~23 MB
- GUI boots and runs entirely in RAM
- Idle RAM with GUI: ~50–100 MB
- With a full browser (Firefox or Chromium) open: usually 300–600 MB total (allocate 512 MB–1 GB to the VM and it's smooth)
- Perfect for "minimal" + "less RAM consuming graphical user interface"

Quick setup guide (tested concept — takes 15–30 minutes):

1. On your host (any Linux), install QEMU/KVM + virt-manager (or use VirtualBox).
2. Download TinyCore ISO from tinycorelinux.net (choose the "TinyCore" 64-bit version).
3. Create a new VM:
 - 1–2 CPU cores
 - 512 MB–1 GB RAM (it will use far less)
 - No hard disk needed (runs from RAM) or a tiny 1–2 GB virtual disk if you want persistence
 - Boot the ISO
4. Inside the running TinyCore:
 - Use the App Browser (tce) to install: firefox or chromium, x11vnc (or tightvncserver), and a tiny window manager if you want one (FLWM is already there).
 - Run the browser in kiosk/fullscreen mode pointing at your target site.
 - Start x11vnc or tightvncserver (one command).

5. From your host, connect with any VNC viewer (or noVNC in a browser for web access).

You now have: separate kernel, separate frame buffer/rendering, VNC output, extremely low RAM, and full isolation. Malware stays inside the VM. Snapshots make every session disposable.

Even easier alternative if you want a ready desktop: Puppy Linux (any recent variant like Fossapup or BionicPup).

- Runs completely in RAM
- ~150–300 MB total usage with browser
- Pre-built GUI, super simple to install Firefox/Chromium + VNC server
- Slightly heavier than TinyCore but still tiny and beginner-friendly.

Other options (ranked by isolation strength)

Isolation Type	Kernel Separate?	RAM Usage (approx.)	Ease	Best For	Drawback
Chrome/Edge site isolation	No	Very low	Easiest	Daily use	Escape possible
Docker container (neko, Kasm Workspaces, SupraRBI-VNC)	No (shared)	Low–medium	Very easy	Quick web access	Same kernel risk
TinyCore/Puppy VM + VNC	Yes	Extremely low	Medium	Your exact project	Manual VNC setup
Commercial cloud RBI (Symantec, Zscaler, etc.)	Yes	N/A (cloud)	Easiest	Enterprises	Not self-hosted

Extra tips for maximum security in your setup

- Make the VM snapshot-based or disposable (delete/recreate after each use).
- Disable clipboard/file sharing in the hypervisor unless you need it.
- Block downloads inside the guest browser or redirect them to a shared folder you control.

- Use a minimal VNC server with TLS (x11vnc + stunnel or noVNC + nginx).
- Run the VM on a dedicated low-power machine or behind a firewall for extra safety.

This exact VM + VNC approach is what many security researchers and small teams use to replicate commercial RBI locally. It gives you the "separate kernel, render buffer, frame, VNC output" you described, with way less RAM than a normal Ubuntu/Fedora GUI VM.

If you tell me your host OS (Linux/Windows?) or how much RAM your host has, I can give exact commands or a ready-made VM image link. TinyCore will solve the "can't find low-RAM GUI VM" problem — it's literally built for this. Let me know if you hit any step!